RELATÓRIO TÉCNICO: ANÁLISE DE ESTRUTURAS DE DADOS E BALANCEAMENTO

Disciplina: Estrutura de Dados

Acadêmicos: Ronaldo Dranski, Nefertiti Kruger, Marcos Cardozo

1. INTRODUÇÃO

Este relatório descreve a implementação e a análise de desempenho das estruturas de dados: Árvore Binária de Busca (BST), Árvore AVL e Árvore Rubro-Negra. O objetivo principal é verificar como o balanceamento automático impacta a eficiência das operações em diferentes volumes de dados. Também apresenta uma solução heurística para o Problema do Caixeiro-Viajante utilizando a lógica do Vizinho Mais Próximo.

2. FUNDAMENTAÇÃO TEÓRICA

2.1. Árvore BST

A BST é uma estrutura onde cada nó possui no máximo dois filhos. A regra de inserção determina que valores menores que o nó pai fiquem à esquerda e valores maiores à direita. Sua principal vantagem é a simplicidade, porém, sua eficiência é comprometida caso os dados sejam inseridos de forma ordenada, o que gera uma árvore degenerada, similar a uma lista ligada.

2.2. Árvore AVL

A AVL é uma árvore balanceada. Ela utiliza o Fator de Balanceamento, diferença de altura entre subárvores, para detectar desequilíbrios. Caso o fator atinja valores fora do intervalo [-1, 1], são executadas rotações simples ou duplas para restaurar a altura logarítmica.

2.3. Árvore Rubro-Negra

Diferente da AVL, a Rubro-Negra utiliza um sistema de cores, vermelho e preto, e regras de propriedade para garantir o balanceamento. Embora não seja tão rígida quanto a AVL, ela garante que nenhum caminho da raiz até uma folha seja mais que o dobro do comprimento de outro, sendo extremamente eficiente para inserções frequentes.

2.4. Problema do Caixeiro-Viajante

O TSP busca o menor caminho que percorra um conjunto de nós e retorne à origem. Por ser um problema NP-Difícil, utilizou-se a heurística do Vizinho Mais Próximo, que escolhe o nó vizinho mais curto a cada passo, priorizando a execução rápida.

3. ANÁLISE ASSINTÓTICA

Abaixo, a tabela comparativa de complexidade para as operações de pior caso:

Algoritmo/Estrutura	**Inserção**	**Busca**	**Remoção**
BST	*O(n)*	*O(n)*	*O(n)*
AVL	*O(log n)*	*O(log n)*	*O(log n)*
Rubro-Negra	*O(log n)*	*O(log n)*	*O(log n)*
TSP (Vizinho Mais Próximo)	*O(n^2)*	-	-

4. METODOLOGIA EXPERIMENTAL

Para a coleta de dados, utilizou-se a linguagem **Java**. O ambiente de testes consistiu em:

- **Hardware:** Intel i5-4210U, 8GB RAM.
- **Software:** Java JDK 17 / JVM.
- **Procedimento:** Foram conduzidas duas baterias de testes distintas para observar o comportamento das árvores em diferentes ordens de grandeza:
    - **Cenário de Médio Porte (Base Projeto II):** Entradas de 1.000, 5.000 e 10.000 elementos.
    - **Cenário de Pequeno Porte (Base Projeto III):** Entradas de 100, 500 e 1.000 elementos.

Consistência Estatística: Cada experimento foi repetido **30 vezes** de forma independente. Utilizou-se o método `System.nanoTime()` para capturar o tempo de execução, garantindo precisão em nanossegundos, posteriormente convertidos para milissegundos para facilitar a leitura.

5. RESULTADOS E DISCUSSÃO

Os experimentos foram divididos em testes de ordenação, busca e eficiência de balanceamento, conforme os dados obtidos abaixo:

Tabela 1: Benchmark de Busca (Tempo Médio em nanossegundos)

Tamanho (N)	**Busca Sequencial**	**Busca Binária**	**Busca em Árvore**
1.000	**648.666,66**	**169.463,33**	**114.660,00**
5.000	**7.313.090,00**	**276.470,00**	**84.473,33**
10.000	**10.191.546,66**	**43.053,33**	**53.323,33**

Tabela 2: Eficiência de Balanceamento (Alturas)

Estrutura	**Altura com 5 elementos**	**Altura após Remoção**
BST (Desbalanceada)	5	4
AVL (Balanceada)	3	3
Rubro-Negra (3 el.)	2	-

Tabela 3: Problema do Caixeiro-Viajante (Custo Médio)

Tamanho (Cidades)	**Média do Custo (Heurística)**
5	103,2
10	130,9
20	177,5

6. ANÁLISE DE RESULTADOS

1. **Balanceamento :**

```
###===== COMPARATIVO DE ÁRVORES =====

#### Árvore Binária de Busca (BST)
* **Em ordem:** 10, 20, 30, 40, 50
* **Altura inicial:** 5
* **Operação:** Removendo 30...
* **Após remoção:** 10, 20, 40, 50
* **Altura após remoção:** 4

#### Árvore AVL
* **Em ordem:** 10, 20, 30, 40, 50
* **Altura inicial:** 3
* **Operação:** Removendo 30...
* **Após remoção:** 10, 20, 40, 50
* **Altura após remoção:** 3

#### Testes de Busca (BST)
* **Buscar 40:** true
* **Buscar 99:** false

#### Árvore Rubro-Negra
* **Em ordem:** 5, 10, 20
* **Altura:** 2
* **Buscar 10:** true
* **Após remover 10:** 5, 20
```

Os resultados confirmam a teoria. Ao inserir os valores 10, 20, 30, 40 e 50 em ordem, a **BST** degenerou em uma lista com **altura 5**. Já a **AVL**, através de suas rotações automáticas, manteve a **altura em 3**, provando ser muito mais eficiente para buscas em árvores povoadas.

2. **Busca :**

```
### ===== TESTES DE BUSCA =====

#### Tamanho: 1000
* **Sequencial** -> Média: 648666.6666666666 | Desvio:
1164243.1204673515
* **Binária** -> Média: 169463.33333333334 | Desvio:
249685.81522029277
* **Árvore** -> Média: 114660.0 | Desvio:
```

```
96361.94477074443

#### Tamanho: 5000
* **Sequencial** -> Média: 7313090.0 | Desvio:
1.8742673012288474E7
* **Binária** -> Média: 276470.0 | Desvio:
1183045.050466521
* **Árvore** -> Média: 84473.33333333333 | Desvio:
52330.10563294857

#### Tamanho: 10000
* **Sequencial** -> Média: 1.019154666666666E7 | Desvio:
2.2254725818032954E7
* **Binária** -> Média: 43053.333333333336 | Desvio:
10655.318963889455
* **Árvore** -> Média: 53323.333333333336 | Desvio:
13696.220484336382
```

É interessante notar que para N=10.000, a **Busca Binária**, em array ordenado, apresentou o melhor tempo absoluto: **43.053,33 ns**. Isso ocorre porque o acesso direto ao índice de um array em Java é extremamente otimizado pela JVM, superando levemente a **Busca em Árvore**: **53.323,33 ns**, que exige a navegação entre diferentes ponteiros. Em contraste, a **Busca Sequencial** demonstrou ser ineficiente para este volume de dados, atingindo a marca de **10.191.546,66 ns**.

3. **Ordenação:**

```
### ===== BENCHMARK DE ORDENAÇÃO =====

#### Tamanho: 100
* **Bubble Médio:** 135.36666666666667
* **Quick Médio:** 57.53333333333333
* **Bubble Pior:** 109.96666666666667
* **Quick Pior:** 17.166666666666668
* **Bubble Melhor:** 115.9
* **Quick Melhor:** 11.066666666666666

#### Tamanho: 500
* **Bubble Médio:** 556.1
* **Quick Médio:** 53.833333333333336
* **Bubble Pior:** 453.93333333333334
* **Quick Pior:** 31.733333333333334
* **Bubble Melhor:** 138.33333333333334
* **Quick Melhor:** 22.166666666666668
```

```
#### Tamanho: 1000
* **Bubble Médio:** 7903.966666666666
* **Quick Médio:** 117.06666666666666
* **Bubble Pior:** 1905.1333333333334
* **Quick Pior:** 160.4
* **Bubble Melhor:** 577.9
* **Quick Melhor:** 35.56666666666667
```

O **QuickSort** superou o **BubbleSort** conforme N cresceu. Em N=1.000, o BubbleSort Médio levou 7.903ms contra apenas 117ms do QuickSort, demonstrando a diferença prática entre complexidade *O(n^2) e O(n log n)*.

4. **Caixeiro-Viajante:**

```
### ===== CAIXEIRO VIAJANTE =====

* **Tamanho 5:** Média do custo: 103.2
* **Tamanho 10:** Média do custo: 130.93333333333334
* **Tamanho 20:** Média do custo: 177.56666666666666
```

A heurística do vizinho mais próximo mostrou um crescimento de custo sublinear em relação ao número de cidades, indicando que a solução encontra caminhos viáveis rapidamente, mesmo que não sejam os ótimos globais.

7. DISCUSSÃO DOS RESULTADOS

A análise dos dados coletados demonstra uma correlação direta entre a fundamentação teórica e o desempenho empírico observado nos testes.

7.1. Eficiência de Balanceamento e Estrutura de Árvores

Os resultados confirmam a teoria estudada. Ao inserir os valores 10, 20, 30, 40 e 50 em ordem crescente, a BST degenerou completamente, tornando-se uma lista linear com altura 5. Já a Árvore AVL, através de suas rotações automáticas, manteve a altura em 3, provando ser muito mais eficiente para buscas em árvores povoadas. A Árvore Rubro-Negra também apresentou excelente desempenho, mantendo o equilíbrio necessário para operações rápidas.

7.2. Performance nos Algoritmos de Busca

A análise dos tempos em nanossegundos revelou diferenças críticas entre as abordagens. A Busca Sequencial demonstrou ser inviável para grandes volumes, atingindo a marca de **10.191.546,66 ns** para **10.000** elementos. Em contrapartida, a Busca Binária obteve o melhor tempo absoluto no cenário de **10.000** elementos: **43.053,33 ns**, superando levemente a Busca em Árvore: **53.323,33 ns**. Isso ocorre porque o acesso ao índice de um array em Java é extremamente veloz, enquanto a árvore exige a navegação entre referências de memória.

7.3. Ordenação e Escalabilidade

O benchmark de ordenação evidenciou a superioridade do QuickSort sobre o BubbleSort. Enquanto o BubbleSort levou **7.903,96 ms** para ordenar elementos, o QuickSort realizou a mesma tarefa em apenas **117,06 ms**. Essa discrepância confirma a diferença prática entre a complexidade quadrática *O(n^2)* e a logarítmica *O(n log n)*, mostrando que algoritmos simples como o BubbleSort não escalam para sistemas reais.

7.4. Otimização no Problema do Caixeiro-Viajante (TSP)

A heurística do Vizinho Mais Próximo mostrou um crescimento de custo controlado em relação ao número de nós (variando de 103,2 a 177,5). Embora não garanta o caminho ótimo global, ela se mostrou uma solução eficaz para encontrar trajetos viáveis em tempo de execução reduzido, validando o uso de algoritmos gulosos para problemas de alta complexidade.

8. CONCLUSÃO

A realização deste projeto permitiu validar, por meio de experimentação prática, que a escolha da estrutura de dados e do algoritmo é o fator determinante para a performance de qualquer software. As árvores auto-balanceadas (AVL e Rubro-Negra) provaram sua importância ao evitar que a estrutura degradê para uma lista simples, mantendo a eficiência mesmo com dados inseridos de forma ordenada.

Ficou demonstrado que, para grandes volumes de dados, métodos de divisão e conquista como o QuickSort e a Busca Binária são indispensáveis. Além disso, a aplicação da heurística no TSP evidenciou como estratégias de otimização podem simplificar problemas matematicamente complexos. Em suma, o trabalho reforça que entender a complexidade algorítmica é essencial para o desenvolvimento de soluções computacionais escaláveis e eficientes.